

Name : Fahad Ali

Roll no : 23P-0569

Section : CS-4K

Operating system

Lab 4 Task:

Task:1

Q No: 1

23P-0569

```
#include <stdio.h>
#include <unistd.h>
int main () {
    int a = 1;
    fork();
    a += 1;
    fork();
    a += 2;
    printf("PID: %d, a = %d\n", getpid(), a);
    return 0;
}
```

①

So, we have initially process 1 and After first fork $\rightarrow$ 2 process and after 2nd ~~process~~ fork() $\rightarrow$ each of process create one one process
So, there are 4 total process are created

②

each process run remaining code so printf runs 4 time

③

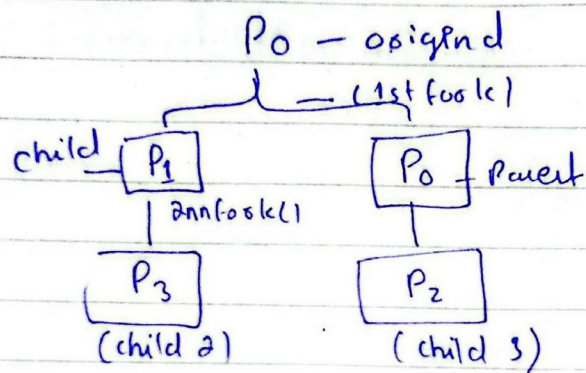
initially $a = 1$

After first fork() $a = 2$

After 2nd fork() $a = 4$

So total change is $= 4$

④



⑤

No, Because os scheduler execution order
So, process run independently.

Task : 2

23P-0569

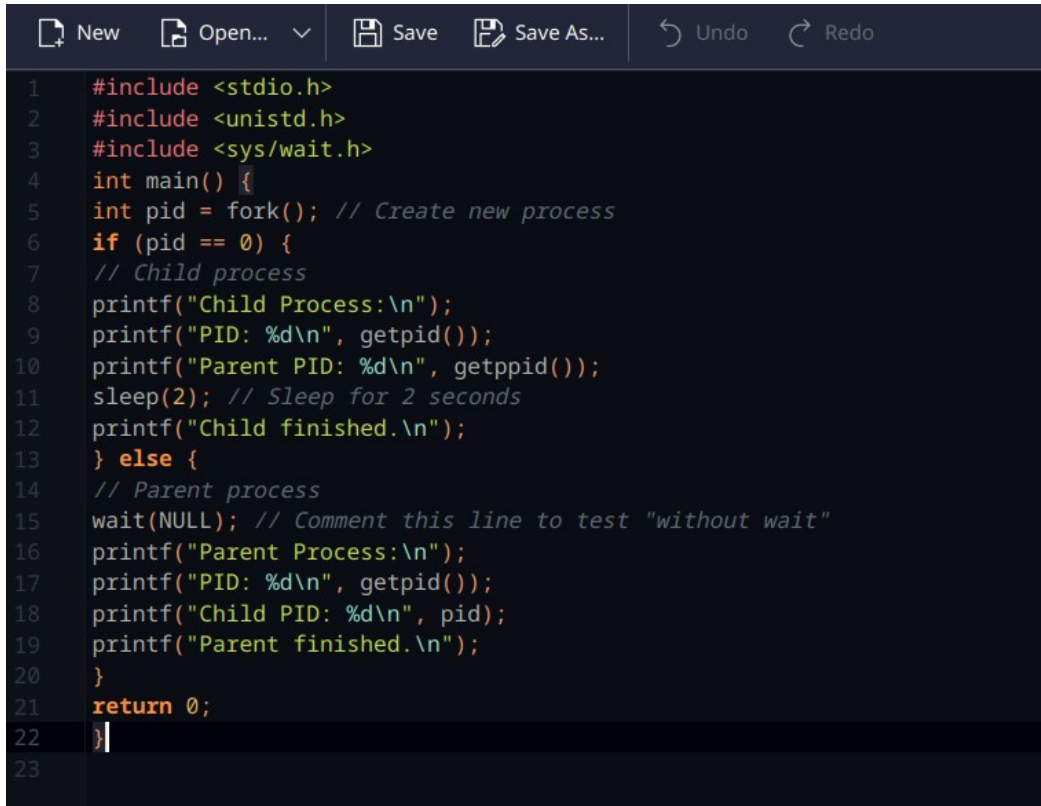
Q No: 2 _____

- (a) `exec()` replace current process with new program
- (b) Before replacing the Process
Hello from `exec`!
- (c) Because `exec()` replace the process
`printf("After exec...")` will not if `exec()` execute successfully.
- (d) if Path (bin/echo) was incorrect the
`exec()` return -1 and does not
~~execute~~ ~~ex~~ replace process.
- (e) After removing `exec()` the output
will sequence wise.
 - Before replacing the process
 - Hello from `exec`
 - After `exec` - will this line run

Task:3

With wait():

In this Parent waits for child to finish....



```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4  int main() {
5      int pid = fork(); // Create new process
6      if (pid == 0) {
7          // Child process
8          printf("Child Process:\n");
9          printf("PID: %d\n", getpid());
10         printf("Parent PID: %d\n", getppid());
11         sleep(2); // Sleep for 2 seconds
12         printf("Child finished.\n");
13     } else {
14         // Parent process
15         wait(NULL); // Comment this line to test "without wait"
16         printf("Parent Process:\n");
17         printf("PID: %d\n", getpid());
18         printf("Child PID: %d\n", pid);
19         printf("Parent finished.\n");
20     }
21     return 0;
22 }
23
```

```
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy
fkhokar@fedora:~$ cd Desktop
fkhokar@fedora:~/Desktop$ gedit fork.c
fkhokar@fedora:~/Desktop$ gcc fork.c
fkhokar@fedora:~/Desktop$ ./a.out
Child Process:
PID: 19355
Parent PID: 19354
Child finished.
Parent Process:
PID: 19354
Child PID: 19355
Parent finished.
fkhokar@fedora:~/Desktop$
```

When wait() was removed, the parent did not wait for the child to finish so the both processes run at the same time and the output order became random.

Without wait():

Parent does not wait....

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int pid = fork(); // Create new process
    if (pid == 0) {
        // Child process
        printf("Child Process:\n");
        printf("PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        sleep(2); // Sleep for 2 seconds
        printf("Child finished.\n");
    } else {
        // Parent process
        //wait(NULL); // Comment this line to test "without wait"
        printf("Parent Process:\n");
        printf("PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent finished.\n");
    }
    return 0;
}
```

```
fkhoar@fedora:~/Desktop$ gcc fork.c
fkhoar@fedora:~/Desktop$ ./a.out
Parent Process:
PID: 20425
Child PID: 20426
Parent finished.
Child Process:
PID: 20426
Parent PID: 20425
fkhoar@fedora:~/Desktop$ Child finished.
□
```

When wait() was removed, the parent didn't wait for the child, so both ran at the same time and the output order became random